Events in JS

In JavaScript, an event is an action or occurrence that the browser can detect, such as clicking a button, pressing a key, typing in an input, submitting a form, or loading a webpage.

For example, common events are `click`, `mouseover`, `keydown`, `input`, `change`, and `submit`.

1. Mouse events

Event	When it occurs
`click`	Element is clicked
`dblclick`	Element is double-clicked
`mousedown`	Mouse button is pressed
`mouseup`	Mouse button is released
`mousemove`	Mouse moves over an element
`mouseover`	Mouse enters an element or its descendant
`mouseout`	Mouse leaves an element or moves to a descendant
`mouseenter`	Mouse enters an element
`mouseleave`	Mouse leaves an element
`contextmenu`	Right-click context menu is requested

2. Keyboard events

Event	When it occurs
`keydown`	A key is pressed
`keyup`	A key is released

Note: `keypress` is deprecated. Prefer `keydown` and `keyup`.

3. Form and input events

Event	When it occurs
`input`	Input value changes as the user types
`change`	A value is committed or changed
`submit`	A form is submitted
`reset`	A form is reset
`focus`	An element receives focus
`blur`	An element loses focus
`focusin`	An element or its descendants receive focus
`focusout`	An element or its descendants lose focus
`invalid`	A form control fails validation
`select`	Text is selected in an input or textarea

Remember the difference:

- `input` — usually fires while typing.
- `change` — fires when a change is committed; exact timing depends on the control.
- `submit` — fires when a form is submitted.

4. Window and page events

Event	When it occurs
`load`	Page and dependent resources finish loading
`DOMContentLoaded`	HTML document has been parsed
`beforeunload`	Document is about to be unloaded
`unload`	Document is unloading; avoid for important cleanup
`pageshow`	Page is shown, including from browser cache
`pagehide`	Page is being hidden or left
`resize`	Window size changes
`scroll`	Document or element is scrolled
`hashchange`	URL fragment (`#...`) changes
`popstate`	Active history entry changes

Event	When it occurs
`online`	Browser reports network connectivity
`offline`	Browser reports loss of network connectivity
`visibilitychange`	Document visibility changes
`error`	An error occurs, such as a resource-loading error
`abort`	A resource load is aborted
`pageswap`	A page swap occurs in supported navigation scenarios
`pagereveal`	A page is revealed in supported navigation scenarios

5. Drag-and-drop events

Event	When it occurs
`dragstart`	Dragging begins
`drag`	Element is being dragged
`dragend`	Dragging ends
`dragenter`	Dragged item enters a target
`dragleave`	Dragged item leaves a target
`dragover`	Dragged item moves over a target
`drop`	Item is dropped on a target

6. Pointer events

Pointer events work with mouse, touch, and pen devices through a common event system.

MDN+1

Event	When it occurs
`pointerover`	Pointer moves over an element
`pointerenter`	Pointer enters an element
`pointerdown`	Pointer becomes active
`pointermove`	Pointer moves
`pointerup`	Pointer is released

Event	When it occurs
`pointercancel`	Pointer interaction is canceled
`pointerout`	Pointer moves out of an element
`pointerleave`	Pointer leaves an element
`gotpointercapture`	Element captures a pointer
`lostpointercapture`	Element releases pointer capture

7. Touch events

Mostly useful for touch-enabled devices.

Event	When it occurs
`touchstart`	Finger touches the screen
`touchmove`	Finger moves on the screen
`touchend`	Finger is removed from the screen
`touchcancel`	Touch interaction is canceled

8. Clipboard events

Event	When it occurs
`copy`	User copies content
`cut`	User cuts content
`paste`	User pastes content

9. Wheel events

Event	When it occurs
`wheel`	Mouse wheel or similar pointing-device control moves

10. Media events (audio and video)

Event	When it occurs
`loadstart`	Media loading starts

Event	When it occurs
`progress`	Browser fetches media data
`suspend`	Browser suspends media fetching
`abort`	Media loading is aborted
`error`	Media loading fails
`emptied`	Media element becomes empty
`stalled`	Media data stops arriving unexpectedly
`loadedmetadata`	Media metadata is available
`loadeddata`	Data for the current playback position is available
`canplay`	Media can start playing
`canplaythrough`	Browser estimates media can play through without buffering
`durationchange`	Media duration changes
`play`	Playback is requested to start
`playing`	Playback actually starts or resumes
`pause`	Playback is paused
`waiting`	Playback waits for more data
`seeking`	Seeking starts
`seeked`	Seeking finishes
`timeupdate`	Playback position updates
`ended`	Playback reaches the end
`ratechange`	Playback speed changes
`volumechange`	Volume or muted state changes
`cuechange`	Active text-track cue changes
`encrypted`	Encrypted media data is encountered
`waitingforkey`	Playback waits for a decryption key

11. CSS animation events

Event	When it occurs
`animationstart`	CSS animation starts

Event	When it occurs
`animationiteration`	An animation iteration completes
`animationend`	Animation finishes
`animationcancel`	Animation is canceled

12. CSS transition events

Event	When it occurs
`transitionrun`	Transition is created
`transitionstart`	Transition begins running
`transitionend`	Transition finishes
`transitioncancel`	Transition is canceled

13. Text composition events

Useful for languages, virtual keyboards, and input methods.

Event	When it occurs
`compositionstart`	Text composition starts
`compositionupdate`	Composed text changes
`compositionend`	Text composition finishes

14. Other useful browser events

Event	When it occurs
`beforeinput`	Just before an input edit is applied
`formdata`	A form's submission data is constructed
`toggle`	A supported disclosure or popover changes state
`beforetoggle`	A supported disclosure or popover is about to change state
`fullscreenchange`	Fullscreen state changes
`fullscreenerror`	Fullscreen request fails

Event	When it occurs
`securitypolicyviolation`	Content Security Policy is violated
`selectionchange`	Text selection or cursor selection changes
`slotchange`	Assigned nodes of a Web Component slot change
`cancel`	A cancelable UI element, such as a dialog, is canceled
`close`	A supported element, such as a dialog, closes
`scrollend`	Scrolling has finished
`command`	A supported command-based UI action is invoked
`contentvisibilityautostatechange`	Automatic content visibility state changes

15. Browser storage, network and communication events

Event	When it occurs
`storage`	Relevant storage changes in another document/context
`message`	A message is received
`messageerror`	A message cannot be deserialized
`open`	A WebSocket connection opens
`close`	A WebSocket connection closes
`online`	Browser reports connectivity restored
`offline`	Browser reports connectivity lost
`unhandledrejection`	A Promise rejection has no handler
`rejectionhandled`	A handler is added to a previously unhandled rejection
`readystatechange`	Document or request ready state changes
`loadend`	An asynchronous request or read operation finishes
`timeout`	A supported operation times out

16. File input and file-reading events

Event	When it occurs
`change`	User selects or changes a file
`loadstart`	File reading starts
`progress`	File-reading progress updates
`load`	File reading succeeds
`error`	File reading fails
`abort`	File reading is aborted
`loadend`	File reading finishes, successfully or otherwise

17. Other specialized events

These are useful in advanced web development and depend on the API or device being used.

Category	Event names
Fullscreen	`fullscreenchange`, `fullscreenerror`
Printing	`beforeprint`, `afterprint`
Gamepad	`gamepadconnected`, `gamepaddisconnected`
Device orientation	`deviceorientation`, `devicemotion`
Geolocation	`geolocation` is not a standard DOM event; geolocation normally uses callbacks or promises
WebRTC	`icecandidate`, `connectionstatechange`, `datachannel`, `track`
IndexedDB	`success`, `blocked`, `upgradeneeded`, `versionchange`, `complete`, `abort`
Service workers	`install`, `activate`, `fetch`, `message`
Web animations	`finish`, `cancel`, `remove`
Web Components	`connectedCallback` and `disconnectedCallback` are lifecycle callbacks, not DOM events
Custom events	Any application-defined event name, such as `cartupdated`

These specialized events do not all belong to ordinary HTML elements. They are associated with particular APIs, objects, or components. For the broader, maintained reference, see MDN's DOM Events documentation.

MDN+1

18. How to use an event in JavaScript

There are different ways to attach an event handler. The recommended general approach is `addEventListener()`.

```
button.addEventListener("click", function () {
    console.log("Button clicked");
});
```

Here:

- `button` — the element listening for the event.
- `"click"` — the event name.
- `function () { ... }` — the code that executes when the event occurs.

You can use the same approach with `input`, `keydown`, `submit`, `mouseover`, and many other event types.